# TRINITY — COMPLETE NOTES IN HINGLISH

## 1. Trinity kya hai?

Trinity ek meta-tooling system hai jo AI models ki capability test karne ke liye difficult evaluation tasks banata hai, unki quality check karta hai aur unki difficulty ko evidence ke through measure karta hai.

Iska main goal hai ensure karna ki tasks genuinely challenging hon aur AI models improve hone ke baad bhi unki difficulty ko dobara evaluate kiya ja sake.

### Trinity ke 5 components

1. **ENGRAM:** Memory maintain karta hai.
2. **FORGE:** Tasks create karta hai.
3. **CRUCIBLE:** Tasks ka audit karta hai.
4. **GAUNTLET:** Artifacts ko challenge karta hai aur gatekeeping mein help karta hai.
5. **MAESTRO:** Poore workflow ko coordinate karta hai.

**Main principle:** Difficulty ko sirf claim nahi karna, evidence se measure karna hai.

---

## 2. ENGRAM — Memory Instrument

ENGRAM ka kaam pichhle tasks, experiments aur evaluation results ki information yaad rakhna hai.

### Main functions

- Historical evidence maintain karna.
- Difficult tasks banane ki useful techniques track karna.
- Outdated information ko identify karna.
- FORGE aur CRUCIBLE ko direction dena.
- New evaluation results ko memory mein add karna.

### Important files

- `ENGRAM.md` — ENGRAM ke rules.
- `.memory/` — ENGRAM ka workbench.
- `DIRECTIVE.md` — ENGRAM ki root report.

**Example:** Agar pichhle experiments mein complex debugging tasks AI ke liye difficult nikle, toh ENGRAM is information ko record karega, taaki future task creation mein iska use ho sake.

**Yaad rakh:** ENGRAM = Remember and Guide.

---

# 3. FORGE — Task Creator

FORGE ka kaam AI models ko test karne ke liye evaluation tasks banana hai.

## Main functions

- Evaluation tasks create karna.
- Project ke instructions follow karna.
- Tasks ko batches mein prepare karna.
- Initial calibration ke liye anchor tasks banana.
- Tasks ko independent auditing ke liye prepare karna.

## Batch structure

Trinity guide ke according, FORGE ek batch mein:

- 30 ordinary task slots
- 1 additional anchor task

handle karta hai.

Anchor task difficulty calibration ke liye hota hai, jabki ordinary tasks regular production ka part hote hain.

## Important files

- `FORGE.md` — FORGE ke rules.
- `.seed/` — FORGE ka workbench.
- `EDICT.md` — FORGE ki root report.

**Example:** FORGE ek multi-file coding problem banata hai jisme AI ko bugs identify karke code fix karna hai.

**Yaad rakh:** FORGE = Create Tasks.

---

# 4. CRUCIBLE — Independent Auditor

CRUCIBLE ka kaam FORGE ke banaye hue tasks ko independently check karna hai.

### Main functions

- Task quality check karna.
- Instructions aur requirements verify karna.
- Defects aur missing evidence identify karna.
- Harness aur relevant artifacts ka audit karna.
- Appropriate disposition issue karna.

### Important files

- `CRUCIBLE.md` — CRUCIBLE ke rules.
- `.audit/` — CRUCIBLE ka workbench.
- `VERDICT.md` — CRUCIBLE ki root report.

**Example:** Agar FORGE ka coding task incomplete hai ya required files missing hain, toh CRUCIBLE is problem ko identify karega.

**Yaad rakh:** CRUCIBLE = Inspect and Audit.

---

## 5. FORGE aur CRUCIBLE directly baat kyun nahi karte?

Maan le exam banane wala person examiner ko directly control kar sakta hai. Toh woh exam ko genuinely challenging banane ke bajaye sirf examiner ko satisfy karne ke liye design kar sakta hai.

Is problem ko reward hacking ke concept se relate kar sakte hain.

Isliye Trinity mein FORGE aur CRUCIBLE ke beech direct communication nahi hoti.

ENGRAM unke beech ek controlled bridge ka kaam karta hai.

### FORGE_VIEW

Ye ENGRAM ki memory se generate hone wala controlled view hai, jo FORGE ko task creation ke liye direction deta hai.

### CRUCIBLE_VIEW

Ye CRUCIBLE ko auditing ke liye direction deta hai.

**Important:** CRUCIBLE_VIEW mein hardness contract expose nahi kiya jata, taaki auditor author ke difficulty targets dekhkar apna judgement na banaye.

Dono views complete memory, pinned inputs aur declared evaluation time ke basis par generate hote hain. Inhe record bhi kiya jata hai, taaki dobara calculate aur verify kiya ja sake.

**Yaad rakh:** ENGRAM is the only bridge between FORGE and CRUCIBLE.

---

# 6. GAUNTLET — Gatekeeping Component

GAUNTLET main three instruments mein se ek nahi hai. Ye unke bahar operate karta hai.

## Main functions

- Artifacts ko challenge karna.
- Authorized approval checks mein help karna.
- Required gatekeeping process ko support karna.

## Associated locations

- `.memory/gauntlet/`
- `.seed/gauntlet/`
- `.audit/approvals/`

GAUNTLET ka apna independent fourth workbench nahi hai.

**Yaad rakh:** GAUNTLET = Challenge and Guard.

---

# 7. MAESTRO — Workflow Coordinator

MAESTRO ka kaam ENGRAM, FORGE aur CRUCIBLE ke workflow ko prescribed order mein coordinate karna hai.

Ye khud task creation ya independent auditing ka main kaam nahi karta.

## Main functions

- Instruments ko correct order mein sequence karna.
- Pending approval gates ko consolidate karna.
- Workflow coordination karna.

MAESTRO apni separate workbench, ledger ya root report maintain nahi karta.

**Yaad rakh:** MAESTRO = Coordinate.

---

# 8. Difficulty Tiers

Trinity mein difficulty ke three tiers hain:

## 1. Baseline

Basic reference difficulty level.

## 2. Hard

Baseline se zyada challenging tasks.

## 3. Frontier-defeat

Aise tasks jo advanced frontier-level AI models ko challenge karne ke liye intended hain.

**Important:** Sirf task ko Hard ya Frontier-defeat naam dene se uski difficulty prove nahi hoti. Applicable evaluation evidence zaroori hai.

## SHIP aur SHIP

- **SHIP:** Applicable release conditions satisfy hain aur required evidence release ko support karta hai.
- **SHIP:INFERRED:** Release applicable batch-inferred evidence pathway ke basis par supported hai.

Dono ke evidence pathways alag hain. Inhe equivalent nahi samajhna chahiye.

## Difficulty stale kyun hoti hai?

Aaj jo task difficult hai, future mein improved AI models usse easily solve kar sakte hain.

Isliye difficulty evidence ko time ke saath reassess karna padta hai.

**Yaad rakh:** Difficulty evidence can become stale as AI models improve.

---

# 9. Anchor Tasks

Anchor tasks initial tasks hote hain jinka use regular mass production se pehle difficulty calibrate karne ke liye kiya jata hai.

Trinity guide ke according, FORGE ke first 3–5 tasks anchoring process se guzarte hain.

## Anchor ka purpose

- Difficulty techniques ki practical cost samajhna.
- Task ko solve karne mein lagne wala time measure karna.
- Future task creation ke liye useful calibration evidence collect karna.

### Anchor workflow

1. Requirements aur human-graded reference bundles prepare hote hain.
2. FORGE anchor task banata hai.
3. CRUCIBLE task ka audit karta hai.
4. Harness task execute karta hai.
5. External signer results evaluate aur sign karta hai.
6. ENGRAM evidence ko memory mein add karta hai.
7. Future tasks updated guidance ke according banaye jaate hain.

### Anchor ke statuses

Agar anchor audit mein held ya blocked hai, ya audit ke baad pilot ka wait kar raha hai, toh woh provisional admission ke saath `CANDIDATE` status mein record ho sakta hai.

Candidate anchor quota mein count kar sakta hai, lekin woh `ANCHORED` ke equivalent nahi hai.

Guide ke according, kam se kam 3 anchors ko 2 ya usse zyada axes par `ANCHORED` status tak pahunchna chahiye.

Jab tak required conditions complete nahi hoti, ordinary batches par `HOLD:PILOT_REQUIRED` lag sakta hai.

**Important:** Anchor calibration karta hai; woh apne aap difficulty tier prove nahi karta.

---

## 10. Harness kya hai?

Harness ek evaluation infrastructure hai jo tasks execute karta hai aur results collect karta hai.

### Harness ke possible functions

- Task execution environment prepare karna.
- Declared time budget ke andar tasks run karna.
- Tests execute karna.
- Results aur execution trajectories record karna.
- Task performance evaluate karne mein help karna.

Trinity mein:

- FORGE harness extension generate karta hai.
- CRUCIBLE uske relevant components audit karta hai.

- Gate required configuration pin check karta hai.

Configuration identifier hai `trinity.harness-config/v1`.

### Harness vs Workbench

- `harness/` — Evaluation infrastructure.
- `.memory/` — ENGRAM ka workbench.
- `.seed/` — FORGE ka workbench.
- `.audit/` — CRUCIBLE ka workbench.

**Yaad rakh:** Harness tasks execute karta hai; workbenches instruments ke kaam ko support karte hain.

---

## 11. Trinity ke Operational Lanes

Trinity mein three main operational lanes hain.

### A. Research Lane

Iska kaam memory aur hardness contract establish karna hai.

Activities:

- Hardness contract prepare karna.
- ENGRAM ki memory build aur publish karna.
- FORGE_VIEW aur CRUCIBLE_VIEW generate karna.
- Anchor-standing information maintain karna.

**Sign-off:** Published epoch aur `DIRECTIVE.md`.

### B. Engineering Lane

Iska kaam harness generate aur verify karna hai.

Activities:

- FORGE harness extension generate karta hai.
- CRUCIBLE harness audit karta hai.
- Gate configuration pin verify karta hai.

**Sign-off:** Approved configuration pin aur clean `VERDICT.md`.

Agar fidelity coverage demonstrate nahi hui, toh `HOLD:SUPPRESSED_MEASUREMENT` lag sakta hai.

### C. Operations Lane

Iske two major workflows hain.

**Anchor operations:** Anchors author, audit, execute aur externally evaluate hote hain.

Sign-off: `.memory/anchor_standing.yaml`.

**Ordinary task operations:**

1. Thirty ordinary tasks author karna.
2. Batch seal karna.
3. Har sealed bundle ka audit karna.
4. Results reconcile karna.
5. Bundles ko appropriate directories mein place karna.

Sign-off: `samples/` occupancy aur `EDICT.md`.

**Main principle:** Har stage ko next dependent stage se pehle apni required conditions satisfy karni hoti hain.

---

## 12. SHIP, HOLD aur BLOCK

Ye Trinity ke main dispositions hain.

### SHIP

Applicable conditions satisfy hone par release permitted hai.

### HOLD

Koi required condition pending hai, isliye workflow ko wait karna hoga.

Examples:

- `HOLD:PILOT_REQUIRED` — Required pilot pending hai.
- `HOLD:SUPPRESSED_MEASUREMENT` — Required measurement coverage demonstrate nahi hui.

### BLOCK

Blocking condition ki wajah se release refuse kiya jata hai.

Example:

- `BLOCK:INVALID_PILOT` — Pilot ya executed population ki validity prove nahi ho paayi.

### SHIP

Authorized human ek eligible HOLD ko signed bypass ke through temporarily lift kar sakta hai.

Lekin:

- Bypass active aur applicable hona chahiye.
- Conditional release measured difficulty claim establish nahi karta.
- Bypass apne sunset par expire ho jata hai.
- Bypass kisi BLOCK ko lift nahi kar sakta.

**Easy trick:**

- SHIP = Release
- HOLD = Wait
- BLOCK = Refuse

---

# 13. Important Files aur Directories

## Main contracts

- `ENGRAM.md` — ENGRAM ke rules.
- `FORGE.md` — FORGE ke rules.
- `CRUCIBLE.md` — CRUCIBLE ke rules.
- `GAUNTLET.md` — Gatekeeping instructions.
- `MAESTRO.md` — Workflow coordination instructions.

## Root reports

- `DIRECTIVE.md` — ENGRAM.
- `EDICT.md` — FORGE.
- `VERDICT.md` — CRUCIBLE.

## Workbenches

- `.memory/` — ENGRAM.
- `.seed/` — FORGE.
- `.audit/` — CRUCIBLE.

## Other directories

- `harness/` — Evaluation infrastructure.
- `samples/` — Resident task bundles.

- `delivery/` — Delivery-related bundles.
- `stock/` — Stock-related bundles.
- `trinity/` aur `.trinity/` — Registered repository components.

Guide ke according, total 9 roster-root directories ko apne distinct remotes ke against submodules ke roop mein register karna hota hai, with `branch = main` aur `update = merge`.

### TRACKING.md

Ye ek generated dashboard hai jo workflow progress, outcomes, rework, lane capacity aur health indicators show karta hai.

Isse `pipeline.py render-reports` generate karta hai. Koi instrument directly ismein apna data write nahi karta.

### DEFERRED.md

Ye un contractual obligations ko list karta hai jo tooling automatically enforce nahi karti.

Isliye automated checks pass hone ka matlab ye nahi ki har contractual requirement automatically complete ho gayi hai.

---

## 14. Contracts aur Coding Agents

Trinity ke contracts detailed Markdown instructions hain jo supported coding agents follow karte hain.

Main contracts:

- `ENGRAM.md`
- `FORGE.md`
- `CRUCIBLE.md`

Guide OpenCode, Claude Code, Cursor aur Kiro ke liye command entry points describe karti hai.

`gate.py install` templates se required command doors aur related configuration generate karta hai.

### MCP status tools

1. `gate_status`
2. `pipeline_status`
3. `standing_status`
4. `eta_report`

Ye tools status aur reporting information provide karte hain. Ye independently tasks seal nahi karte, bundles place nahi karte aur dispositions issue nahi karte.

### Gate kya hai?

Gate ek named approval point hai jo SHA-256 digest ke through relevant artifact ya state se bind hota hai.

---

## 15. Repository Integrity aur Firewall

Trinity repository ki integrity aur instruments ki independence maintain karne ke liye rules aur automated checks use karta hai.

### Hardness contract protection

Parent gate `.audit/` ke evidence files scan karta hai aur forbidden hardness-contract references milne par evidence reject karta hai.

### Submodule boundary

`check_parent_layout` verify karta hai ki `.memory/`, `.seed/` aur `.audit/` apne respective remotes ke against submodules ke roop mein registered hain.

### Front-door integrity

Installation checks ensure karte hain ki configured Cursor rules required templates ke saath match karein aur unexpected ya stale rules identify kiye ja sakein.

**Limitation:** Har protection automated nahi hai. Kuch rules contracts aur human procedures par depend karte hain.

---

## 16. Complete Trinity Workflow

Poore workflow ko is order mein yaad kar:

1. **Research:** Hardness contract aur ENGRAM memory establish hoti hai.
2. **Engineering:** Harness generate aur audit hota hai.
3. **Anchoring:** Initial anchor tasks create, audit aur externally evaluate hote hain.
4. **Memory update:** ENGRAM evidence incorporate karta hai.
5. **Task production:** FORGE thirty ordinary tasks ka batch prepare karta hai.
6. **Auditing:** CRUCIBLE sealed bundles audit karta hai.
7. **Routing:** Bundles appropriate directories mein place hote hain.

8. **Monitoring:** Results track hote hain aur stale difficulty evidence reassess hota hai.

Har stage ko applicable gates aur dependencies follow karni hoti hain.

---

# 17. Important Terminology

| Term | Simple Meaning |
|---|---|
| Contract | Instrument ke rules aur instructions |
| Instrument | Trinity ke three main agents mein se ek |
| Workbench | Instrument ka working directory |
| Harness | Evaluation tasks execute karne wala infrastructure |
| Lever | Task ko difficult banane ki reusable technique |
| Tier | Difficulty category |
| Pilot | Frozen task aur frozen solver registry ka external signed evaluation |
| Anchor | Initial calibration task |
| Disposition | Gate ka decision |
| Gate | Named approval point |

| | |
|---|---|
| Epoch | ENGRAM memory aur associated outputs ka published version |
| Stale evidence | Aisa evidence jo intended claim ke liye outdated ho chuka hai |
| Reward hacking | Intended goal ke bajaye evaluation mechanism ko game karna |

---

# 18. Final Quick Revision

**Five components:**

- ENGRAM = Remember.
- FORGE = Create.
- CRUCIBLE = Audit.
- GAUNTLET = Challenge.
- MAESTRO = Coordinate.

**Three difficulty tiers:**

- Baseline.
- Hard.
- Frontier-defeat.

**Three main dispositions:**

- SHIP = Release.
- HOLD = Wait.
- BLOCK = Refuse.

**Three workbenches:**

- `.memory/` = ENGRAM.
- `.seed/` = FORGE.
- `.audit/` = CRUCIBLE.

**Three root reports:**

- `DIRECTIVE.md` = ENGRAM.
- `EDICT.md` = FORGE.
- `VERDICT.md` = CRUCIBLE.

**Sabse important principle:** Trinity ka goal sirf difficult AI tasks banana nahi, balki unhe independently audit karna aur evidence ke through verify karna hai ki unki difficulty meaningful bani hui hai.